CSE 2216: Compiler Laboratory

Lab 07 — Project Report

Implementation of a Mini-Language Compiler (Front-End)

Group Member	Student Role / Primary Part
Shahriar Sourov	Part A — Lexical Analyzer (Flex)
Nukhyai U Marma	Part B — Parser and AST (Bison)
Saad Bin Sultan	Part C — Symbol Table
Abdus Salam Gifari	Part D — Semantic Analysis

Toolchain: Flex 2.6.4 · GNU Bison 3.8.2 · GCC · GNU Make

1. Project Overview

This project implements the front-end of a compiler for a small statically-typed language (the "mini-language"). The front-end takes a source program as input and carries it through four classical phases, reporting errors with line numbers at every stage:

- Lexical analysis — group raw characters into tokens (Flex).
- Syntax analysis and AST construction — validate the token stream against a grammar and build an Abstract Syntax Tree of real node objects (Bison).
- Symbol table construction — record every declared identifier with its type, scope, declaration line, and initialization status.
- Semantic analysis — walk the AST and detect meaning-level errors such as undeclared variables, duplicate declarations, and type mismatches.

The mini-language supports four data types (int, float, string, bool), variable declarations with optional initialization, arithmetic and logical expressions, and two control-flow constructs (if/else and do/while), plus a print statement.

1.1 Build and Run

The project is built with a single Makefile that knows the correct generation order (Bison, then Flex, then GCC):

```
make            # build the compiler  ->  ./minilang
make test       # build, then run on a sample program
make lexer      # build the Part A standalone token dumper
make clean      # remove all generated / compiled files

./minilang tests/01_valid_arithmetic.ml    # run on one source file
./run_tests.sh                             # run on all 12 test programs
```

Requirements: flex, bison, gcc, and make. On Ubuntu/WSL: sudo apt install flex bison build-essential.

1.2 Processing Pipeline

Each phase consumes the output of the previous one:

```
source.ml
   |
   v
 [ LEXER ]   --> token stream        (Part A, lexer.l)
   |
   v
 [ PARSER ]  --> Abstract Syntax Tree (Part B, parser.y + ast.c)
   |
   v
 [ SYMBOL TABLE ]                     (Part C, symtab.c)
   |
   v
 [ SEMANTIC ANALYSIS ] --> verdict    (Part D, semantic.c)
```

1.3 Source Files

File	Phase	Purpose
tokens.h	A	Token codes for the standalone lexer test.
lexer.l	A	Flex specification: regex rules to tokens.
lextest.c	A	Standalone driver that prints the token stream.
ast.h / ast.c	B	AST node types, constructors, and pretty-printer.
parser.y	B	Bison grammar, AST construction, error reporting, main().

File	Phase	Purpose
symtab.h / symtab.c	C	Scope stack and symbol-table entries.
semantic.h / semantic.c	D	AST walk that builds the table and checks semantics.
Makefile	-	Build recipe for the whole project.
tests/	-	12 test programs (valid and each error class).

2. Part A — Lexical Analyzer

The lexer is written in Flex. It reads the source character by character and returns one token per call to yylex(). Whitespace and comments are skipped; newlines are counted so every token carries an accurate line number.

2.1 Recognized Tokens

Category	Tokens
Keywords	int, float, string, bool, if, else, do, while, print
Boolean literals	true, false
Identifiers	letter or underscore, then letters / digits / underscores
Integer literals	one or more digits (e.g. 42)
Float literals	digits, dot, digits (e.g. 3.14)
String literals	double-quoted text on a single line
Arithmetic ops	+ - * / %
Relational ops	== != < <= > >=
Logical ops	&& || !
Assignment	=
Punctuation	( ) { } ; ,
Comments (skipped)	// single-line and /* block */

2.2 Key Design Points

- Longest-match rule: Flex always takes the longest matching lexeme, so >= is one token (GE), not > followed by =, and 3.14 is one float, not 3 . 14.
- First-rule-wins on ties: keyword rules are written before the identifier rule, so int becomes the keyword KW_INT rather than an identifier.
- Automatic line tracking via %option yylineno; newlines inside skipped block comments are still counted, keeping line numbers correct.
- A single lexer source serves both phases: compiled with -DLEX_STANDALONE it drives the Part A token dump; compiled normally it feeds token values (yylval) and locations (yylloc) to the Bison parser.

2.3 Lexical Errors Detected

Error	Example	Reported as
Illegal / unrecognized character	@ # $	Lexical Error at line N: illegal character '@'
Malformed numeric literal	12abc, 12.3.4	Lexical Error at line N: malformed numeric literal '12abc'
Unterminated string literal	"no closing quote	Lexical Error at line N: unterminated string literal

Error rules are placed after the valid rules; because the malformed pattern is longer than the valid prefix it wins by the longest-match rule. On a lexical error the lexer reports it and continues scanning (it does not return a token), so one run can surface several lexical errors.

3. Part B — Parser and AST Construction

The parser is generated by Bison. It requests tokens from the lexer and checks them against the grammar below. In each rule's action it constructs an AST node, so a successful parse yields the complete tree rooted at the global node. Every node stores its source line for later error reporting.

3.1 Grammar (summary)

```
program    -> stmt_list
stmt_list  -> stmt_list stmt | (empty)
stmt       -> decl_stmt | assign_stmt | if_stmt
            | dowhile_stmt | print_stmt | block
            | error ';'                (panic-mode recovery)
decl_stmt  -> type ID ';' | type ID '=' expr ';'
assign_stmt-> ID '=' expr ';'
print_stmt -> 'print' expr ';'
if_stmt    -> 'if' '(' expr ')' block
            | 'if' '(' expr ')' block 'else' block
dowhile_stmt-> 'do' block 'while' '(' expr ')' ';'
block      -> '{' stmt_list '}'
type       -> 'int' | 'float' | 'string' | 'bool'
expr       -> expr OP expr | '!' expr | '-' expr
            | '(' expr ')' | literal | ID
```

3.2 Grammar Design Decisions

- Braces are mandatory for the bodies of if, else, and do/while. This removes the classic "dangling else" ambiguity (there is no brace-less single statement to attach an else to) and keeps the grammar unambiguous without extra precedence tricks.
- Operator precedence and associativity are declared once (with %left / %right) rather than encoded as many layered rules. Lowest-to-highest: || , && , == != , relational , + - , * / % , then unary ! and unary minus. So a + b * c parses as a + (b * c), and comparisons bind looser than arithmetic.
- Expressions and statements are kept separate; a declaration always starts with a type keyword, which lets the parser decide a statement's shape from its first token.
- The grammar is LALR(1) and compiles with zero shift/reduce and zero reduce/reduce conflicts.

3.3 AST Node Types

The tree is built from a single flexible Node struct; each constructor fills only the fields that apply to that node kind. Node kinds and what each represents:

Node kind	Represents
Program / Block	A sequence of statements (Block introduces a scope).
Decl	A declaration: type, name, optional initializer expression.
Assign	An assignment: target name and value expression.
If	Condition, then-block, optional else-block.
DoWhile	Body block and loop condition.
Print	A single expression to print.
BinaryOp / UnaryOp	An operator with its operand sub-trees.
Int / Float / String / Bool	Literal leaf nodes.
Var	A use of a variable name.

Every node records its line number (captured from Bison's @n location tracking), which is what the symbol-table and semantic phases use to report errors precisely.

3.4 Syntax Errors Detected

- Missing or unexpected tokens (for example a missing semicolon, or a keyword where an expression was expected).
- Unmatched parentheses or braces.
- Malformed expressions and statements (for example int d = ; with no right-hand side).

Reported format: Syntax Error at line N: expected X but found Y. When the parser state allows exactly one token, the message names it precisely (for example, expected { but found print). When many tokens would be legal at that point (for example, after a complete sub-expression both ; and every operator are valid), the message reports unexpected Y rather than listing a long set of alternatives.

3.5 Error Recovery Strategy — Panic Mode

The project uses panic-mode recovery rather than stopping at the first syntax error. This is implemented with Bison's special error token in the rule stmt -> error ';'. When a syntax error occurs, the parser discards tokens until it reaches the next semicolon (a reliable statement boundary), then resumes normal parsing.

Why panic mode: a single run can report several independent syntax errors instead of forcing the programmer to recompile once per error. The semicolon is chosen as the synchronization point because it ends most statements, so recovery resumes cleanly at the start of the next statement with little spurious cascading. Semantic analysis is run only when parsing finishes with no lexical or syntax errors, because a partially built tree after a syntax error would produce unreliable semantic diagnostics.

4. Part C — Symbol Table

The symbol table records every declared identifier. It is organized as a stack of scopes to model nesting: the global scope is level 0, and each { } block pushes a scope one level deeper. Entering a block pushes a new scope; leaving it pops that scope, so its variables go out of view. Lookups search from the innermost scope outward, which lets an inner variable shadow an outer one of the same name.

4.1 Entry Fields

Each symbol-table entry stores exactly the fields required by the specification:

Field	Meaning
Identifier name	The variable's name.
Data type	int, float, string, or bool.
Scope level	0 for global; 1, 2, … for nested blocks.
Declaration line	The line on which the variable was declared.
Initialization status	Whether the variable has been given a value (yes/no).

4.2 Operations

- insertSymbol — add a declaration to the current scope; returns failure if a variable of the same name already exists in that scope (used to detect duplicate declarations).
- lookupCurrentScope — search only the current scope; used for the duplicate-declaration check.
- lookupSymbol — search the current scope, then outward through all enclosing scopes; used when a variable is read or assigned.
- enterScope / exitScope — push and pop scopes as blocks are entered and left.

The table is printed at the end of a successful compile so the declared identifiers, their scopes, and their initialization status are visible for inspection.

5. Part D — Semantic Analysis

Semantic analysis is a single recursive walk of the AST. As it visits declarations it populates the symbol table; as it visits expressions and statements it checks meaning and reports semantic errors. Expression types are computed bottom-up: each sub-expression returns its result type, which its parent uses to validate the operation.

5.1 Errors Detected

#	Error	Requirement	Example message
a	Undeclared variable usage	Required	variable 'c' … undeclared variable 'c'
b	Duplicate declaration	Required	duplicate declaration of variable 'x'
c	Type mismatch in expression	Required	cannot apply '+' to 'bool' operands
d	Assignment type mismatch	Bonus	cannot assign 'string' to variable of type 'int'
e	Scope violation	Bonus	variable 'temp' used outside its scope
f	Uninitialized variable use	Bonus	variable 'total' used before initialization

All three required checks and all three bonus checks are implemented. Scope violation is distinguished from a plain undeclared-variable error by keeping a record of every name that was ever declared: if a lookup fails but the name was declared somewhere, it is reported as used outside its scope; if the name was never declared at all, it is reported as undeclared. When a type error is found in a sub-expression, that sub-expression's type is marked as an error type so the mistake does not cascade into further spurious messages.

5.2 Type Compatibility Rules

These are the rules the analyzer enforces:

Operation	Operand requirement	Result type
+ - * /	both numeric (int or float)	float if either is float, else int
% (modulo)	both int	int
< <= > >=	both numeric	bool
== !=	same category (numeric/numeric, bool/bool, string/string)	bool
&& ||	both bool	bool
! (logical not)	bool	bool
unary -	numeric	same numeric type
assignment / init	same type, OR int value into a float variable (widening)	-

The only implicit conversion allowed is widening an int to a float on assignment or initialization (for example float ok = 7; is valid). Narrowing (float into int) and all cross-category assignments (for example int x = "hi";) are reported as mismatches. Conditions of if and while must be of type bool.

6. Unified Error Reporting Format

All three phases report through the same template, <Phase> Error at line N: <message>, so output is consistent regardless of which phase found the problem:

```
Lexical Error at line 4: unterminated string literal
Syntax Error at line 3: expected { but found print
Semantic Error at line 8: variable 'total' used before initialization
Semantic Error at line 2: type mismatch: cannot assign 'string' to variable of type
'int'
```

Error messages are written to standard error and the AST / symbol-table dumps to standard output; redirecting with 2>&1 interleaves them in source order. Every message carries the line number of the offending construct.

7. Test Programs

Twelve test programs are provided (exceeding the required ten): three valid programs, two with lexical errors, two with syntax errors, and five covering each semantic error class.

Test file	Category	What it exercises
01_valid_arithmetic.ml	Valid	Precedence, %, unary minus, bool expressions.
02_valid_control_flow.ml	Valid	if/else, do/while, nested control flow.
03_valid_scopes.ml	Valid	Nested scopes and variable shadowing.
04_lex_illegal_char.ml	Lexical	Illegal characters @, #, $.
05_lex_bad_literals.ml	Lexical	Malformed numbers and unterminated string.
06_syntax_missing_semicolon.ml	Syntax	Several missing semicolons (panic-mode).
07_syntax_unmatched.ml	Syntax	Unmatched parenthesis; missing if-condition parens.
08_sem_undeclared.ml	Semantic (a)	Use of undeclared variables.
09_sem_duplicate.ml	Semantic (b)	Duplicate declarations in a scope.
10_sem_type_mismatch_expr.ml	Semantic (c)	Type mismatches inside expressions.
11_sem_assign_mismatch.ml	Semantic (d)	Assignment type mismatches (plus one legal widening).
12_sem_scope_uninit.ml	Semantic (e,f)	Scope violation and uninitialized use.

Note: the lexical-error tests also surface syntax errors. This is expected and realistic: once an illegal token is skipped, the surrounding tokens no longer form a valid statement, so the parser reports its own error. Each phase independently catches its own class of problem.

8. Limitations

- The front-end stops after semantic analysis; it does not generate intermediate or target code.
- Semantic analysis runs only when parsing finishes with no lexical or syntax errors, so semantic problems in a file that also has syntax errors are not reported until the syntax errors are fixed.
- There are no functions, arrays, or user-defined types; the language has only scalar variables of the four built-in types.
- String literals must fit on a single line and support no escape sequences; there is no string concatenation operator.
- When many tokens are valid at a syntax-error point, the message reports the found token without enumerating every expected alternative, to avoid unreadably long messages.
- Uninitialized-use detection is flow-insensitive: it checks the initialization flag at the point of use and does not perform full data-flow analysis across branches.
- Integer and float literals are range-checked only as far as the C library routines (atoi / atof) allow; overflow is not separately diagnosed.

9. Team Member Contributions

Member	Part	Contribution
Shahriar Sourov	A	Flex lexer specification: token rules, comment and whitespace handling, line tracking, and all three lexical-error checks; the standalone token-dump driver.
Nukhyai U Marma	B	Bison grammar design, AST node definitions and constructors, location tracking, the panic-mode error-recovery rule, and the syntax-error reporting format.
Saad Bin Sultan	C	Symbol table: the scope-stack data structure, insert/lookup operations, duplicate detection support, and the table dump.
Abdus Salam Gifari	D	Semantic analyzer: the AST walk, type inference and compatibility rules, and all required and bonus semantic checks; integration of the phases in main().

All members jointly prepared the test programs and this report.